

EC2x&EG9x&EG25-G

QuecOpen 数据拨号用户指导

LTE Standard 模块系列

版本: EC2x&EG9x&EG25-G_QuecOpen_数据拨号用户指导_V1.0

日期: 2020-05-31

状态: 受控文件

上海移远通信技术股份有限公司始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233

电话：+86 21 51086236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：

<http://www.quectel.com/cn/support/sales.htm>

如需技术支持或反馈我司技术文档中的问题，可随时登陆如下网址：

<http://www.quectel.com/cn/support/technical.htm>

或发送邮件至：support@quectel.com

前言

上海移远通信技术股份有限公司提供该文档内容用以支持其客户的产品设计。客户须按照文档中提供的规范、参数来设计其产品。由于客户操作不当而造成的人身伤害或财产损失，本公司不承担任何责任。在未声明前，上海移远通信技术股份有限公司有权对该文档进行更新。

版权申明

本文档版权属于上海移远通信技术股份有限公司，任何人未经我司允许而复制转载该文档将承担法律责任。

版权所有 ©上海移远通信技术股份有限公司 2020，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2020.

文档历史

修订记录

版本	日期	作者	变更表述
1.0	2020-05-31	王辉/ 钱润生/ 匡昌胜/ 钱云绿/ 周守亚	初始版本

目录

文档历史	1
目录	2
表格索引	5
图片索引	6
1 引言	7
1.1. 适用模块	7
2 网卡连接	8
3 AP 侧拨号	9
3.1. 设备检查	9
3.2. APN 配置	9
3.3. 单路拨号	10
3.4. 多路拨号	10
3.4.1. 拨号	10
3.4.2. 路由等参数设置	11
3.5. 3GPP2/CDMA 网络拨号	11
4 不同应用场景下的配置	12
4.1. 场景 1: 单路	12
4.2. 场景 2: 单路 + ECM 设备	13
4.3. 场景 3: 多路	14
4.3.1. 规则清空	14
4.3.2. 默认路由	14
4.3.3. 默认 DNS 设置	15
4.3.4. 通过 APN2 访问某个 Server	15
4.3.4.1. DNS 路由配置	15
4.3.4.2. 域名解析	15
4.3.4.3. 路由配置	15
4.4. 场景 4: 多路 + ECM (1)	16
4.5. 场景 5: 多路 + ECM (2)	17
4.5.1. 域名解析	17
4.5.2. 路由设置	18
4.6. 场景 6: 多路 + ECM (3)	18
4.6.1. 规则清空	19
4.6.2. AP 侧路由设置	19
4.6.3. AP 侧 DNS 设置	19
4.6.4. 转发表设置	19
4.6.5. 策略路由设置	20
4.6.6. 上位机 DNS 服务器设置	20
4.6.6.1. dnsmasq 配置	20
4.6.6.2. APN1 的 DNS 路由设置	21
4.7. 场景 7: 多路 + ECM (4)	21

4.7.1.	基本设置	22
4.7.2.	AP 侧通过 APN1 访问某个 Server	22
4.7.2.1.	域名解析	22
4.7.2.2.	路由设置	22
4.7.3.	上位机通过 APN2 访问某个 Server	22
4.7.3.1.	域名解析	22
4.7.3.2.	路由设置	22
4.8.	场景 8: 多路 + ECM (5)	23
4.8.1.	AP 侧路由和 DNS 设置	24
4.8.2.	上位机策略路由设置	24
4.8.3.	DNS 解析设置	24
5	补充说明	25
5.1.	API	25
5.2.	命令	25
5.2.1.	PING	25
5.2.2.	NSLOOKUP	25
6	数据拨号 API	26
6.1.	数据拨号 API 类型	26
6.1.1.	ql_data_call_error_e	26
6.1.2.	ql_data_call_state_e	26
6.1.3.	ql_data_call_ip_family_e	26
6.1.4.	ql_apn_pdp_type_e	26
6.1.5.	ql_apn_auth_proto_e	27
6.1.6.	v4_address_status	27
6.1.7.	v6_address_status	27
6.1.8.	ql_data_call_state_s	27
6.1.9.	ql_data_call_s	28
6.1.10.	pkt_stats	28
6.1.11.	v4_info	28
6.1.12.	v6_info	28
6.1.13.	ql_data_call_info_s	29
6.1.14.	ql_apn_info_s	29
6.1.15.	ql_apn_add_s	29
6.1.16.	ql_apn_info_list_s	30
6.2.	函数	30
6.2.1.	QL_Data_Call_Init	30
6.2.2.	QL_Data_Call_Destroy	30
6.2.3.	QL_Data_Call_Start	31
6.2.4.	QL_Data_Call_Stop	31
6.2.5.	QL_Data_Call_Info_Get	32
6.2.6.	QL_APN_Set	32
6.2.7.	QL_APN_Get	33
6.2.8.	QL_APN_Add	33
6.2.9.	QL_APN_Del	34

6.2.10.	QL_APN_Get_Lists.....	34
6.2.11.	QL_Data_Call_Init_Precondition.....	34
7	常见问题.....	36
7.1.	抓取 PCAP 日志	36
7.2.	查看 iptables 表	36
7.3.	3GPP2/CDMA 因鉴权无法拨号上网	37
7.4.	模块开机拨号失败	37
7.5.	发送 DNS Server IPv6 Address Request 失败.....	37
8	附录 A 参考文档及术语缩写	38

表格索引

表 1: 适用模块 7

表 2: 参考文档 38

表 3: 术语缩写 38

图片索引

图 1: 场景 1 (单路) 12

图 2: 场景 2 (单路 + ECM 设备) 13

图 3: 场景 3 (多路) 14

图 4: 场景 4 (多路 + ECM (1)) 16

图 5: 场景 5 (多路 + ECM (2)) 17

图 6: 场景 6 (多路 + ECM (3)) 18

图 7: 场景 7 (多路 + ECM (4)) 21

图 8: 场景 8 (多路 + ECM (5)) 23

1 引言

移远通信 LTE Standard 模块支持 QuecOpen®方案，本文档主要描述 LTE Standard QuecOpen 模块无线数据业务的建立过程，即数据拨号的过程。整个拨号过程，需严格按照第3章的顺序执行，特别是初次接触无线业务的客户。

1.1. 适用模块

表 1：适用模块

模块系列	模块
EC2x 系列 QuecOpen	EC25 系列 QuecOpen
	EC21 系列 QuecOpen
	EC20 R2.1 QuecOpen
	EC20-CN QuecOpen
EG9x 系列 QuecOpen	EG95 系列 QuecOpen
	EG91 系列 QuecOpen
EG25-G QuecOpen	EG25-G QuecOpen

2 网卡连接

本章节主要介绍 USB-ECM 和 USB-RNDIS 网卡的使用。

- 关于 USB-ECM 网卡连接的使用说明，请参考文档 [1]。
- 关于 USB-RNDIS 网卡连接的使用说明，请参考文档 [2]。

3 AP 侧拨号

本章节介绍的内容是基于 OpenLinux SDK 中包含的 API 来实现的。

3.1. 设备检查

拨号之前需做一系列基本检查，判断模块是否处于基本的正常工作状态，具体步骤如下：

1. 通过 USB 转串口线连接 PC 至模块的主串口或者 USB 端口。
2. 插入(U)SIM 卡和天线，并上电。
3. 通过如下 API 函数，按顺序检查设备状态。
 - a) 检测(U)SIM 卡：QL_MCM_SIM_GetCardStatus()
 - b) 检测信号强度：QL_MCM_NW_GetSignalStrength()
 - c) 检测模块注网：QL_MCM_NW_GetRegStatus()
 - d) 查询运营商：QL_MCM_NW_GetRegStatus()
 - e) 查询网络接入技术：QL_MCM_NW_GetRegStatus()
 - f) 查询拨号服务状态：QL_Data_Call_Init_Precondition()

3.2. APN 配置

一般来说，对于多路拨号的使用场景，经常需要设置一些特殊 APN 用于专网访问。可使用 QL_Apn_Get() 和 QL_Apn_Set() 查询及配置各 APN 的参数，请参考 example_apn_v2.c。

备注

1. 必须在拨号之前进行 APN 参数配置，配置后参数自动保存，重启仍有效。
2. 通常公网 APN 无需设置用户名和密码以及鉴权参数。
3. 通常专网 APN 是否需要设置用户名和密码以及鉴权参数，需要咨询运营商。
4. 针对 CDMA 的 eHRPD 网络下 APN 设置，强制设置 profile_id 为 0，在拨号接口传入 APN 用户名和密码。参数 profile_id 的写入不影响 HRPD 网络的拨号。

3.3. 单路拨号

本章节主要介绍单路拨号的步骤，无需考虑路由、转发表和 DNS 的配置：

1. 设置 `QL_Data_Call_Set_Default_Profile ()` 的值与请求拨号的 `profile_id` 值一致。
2. 参照第 3.4 章执行一次拨号，仅建立一路数据业务通道。

3.4. 多路拨号

3.4.1. 拨号

```
//初始化并注册回调函数。
QL_Data_Call_Init(user_callback)

//不使用自动配置默认路由和默认转发。
QL_Data_Call_Set_Default_Profile (8)

//建立第 1 路数据业务通道。
Int err_code1;
ql_data_call_s data1_call_paras;

data1_call_paras.profile_idx = 1; //第 1 路。
data1_call_paras.ip_family = QL_DATA_CALL_IPV4; //只拨 IPv4。
data1_call_paras.reconnet = true; //打开自动重连。

QL_Data_Call_Start(&data1_call_paras, & err_code1) //当前 Linux 系统会出现 rmnet_data0。

//建立第 2 路数据业务通道。
Int err_code2;
ql_data_call_s data2_call_paras;

data2_call_paras.profile_idx = 2; //第 2 路。
data2_call_paras.ip_family = QL_DATA_CALL_IPV4;
data2_call_paras.reconnet = true;

QL_Data_Call_Start(&data2_call_paras, & err_code2) //当前 Linux 系统会出现 rmnet_data1

//建立第 3 路数据业务通道。
Int err_code3;
ql_data_call_s data3_call_paras;

data3_call_paras.profile_idx = 3; //第 3 路。
```

```
data3_call_paras.ip_family = QL_DATA_CALL_IPV4;
data3_call_paras.reconnet = true;

QL_Data_Call_Start(&data3_call_paras, &err_code3) //当前 Linux 系统会出现 rmnet_data2。
```

备注

多路拨号时需要注意 profile_idx 值无需和 QL_Data_Call_Get_Default_Profile ()值保持一致。

3.4.2. 路由等参数设置

本部分涉及到的不同应用场景，路由等参数的设置步骤有所不同，请参照第4章了解详细信息。

3.5. 3GPP2/CDMA 网络拨号

请按照以下步骤启用 CDMA 网络拨号：

1. 确定所用的(U)SIM 卡运营商是否支持 CDMA 网络。
2. 拨号之前，调用 QL_MCM_NW_GetRegStatu()查询当前模块注册在 LTE 网络还是 CDMA 网络。
3. profile_id 强制设置为 0，同时 CDMA 网络不支持多路拨号。

```
Int err_code1;
ql_data_call_s data1_call_paras;
char username[] = {"this is example"};
char passwd[] = {"this is example"};

data1_call_paras.profile_idx = 0; //profile_idx 必须为 0。
data1_call_paras.ip_family = QL_DATA_CALL_IPV4; //只拨 IPv4。
data1_call_paras.reconnet = true; //打开自动重连。
data1_call_paras.cdma_username = username;
data1_call_paras.cdma_password = passwd;
QL_Data_Call_Start(&data1_call_paras, &err_code1) //当前 Linux 系统会出现 rmnet_data0。
```

备注

当前网络是 HPRD 还是 eHRPD 对 3GPP2/CDMA 网络拨号没有影响。

4 不同应用场景下的配置

4.1. 场景 1：单路

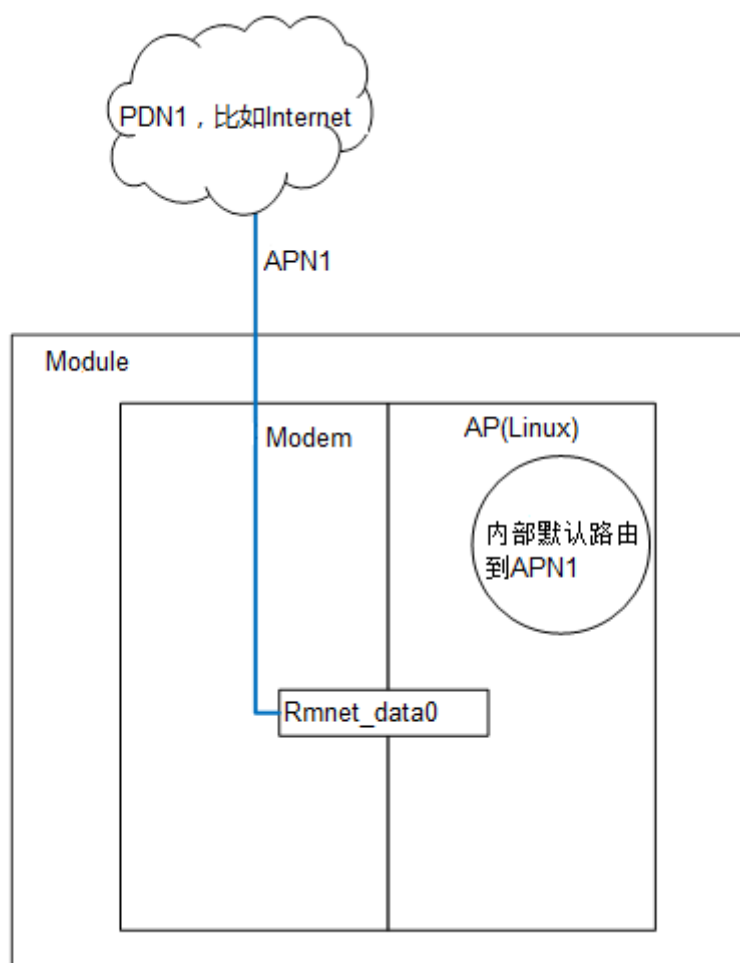


图 1：场景 1（单路）

此应用场景下无需任何路由、转发表和 DNS 配置，仅需按照第3.3 章拨号步骤正确执行即可。

4.2. 场景 2：单路 + ECM 设备

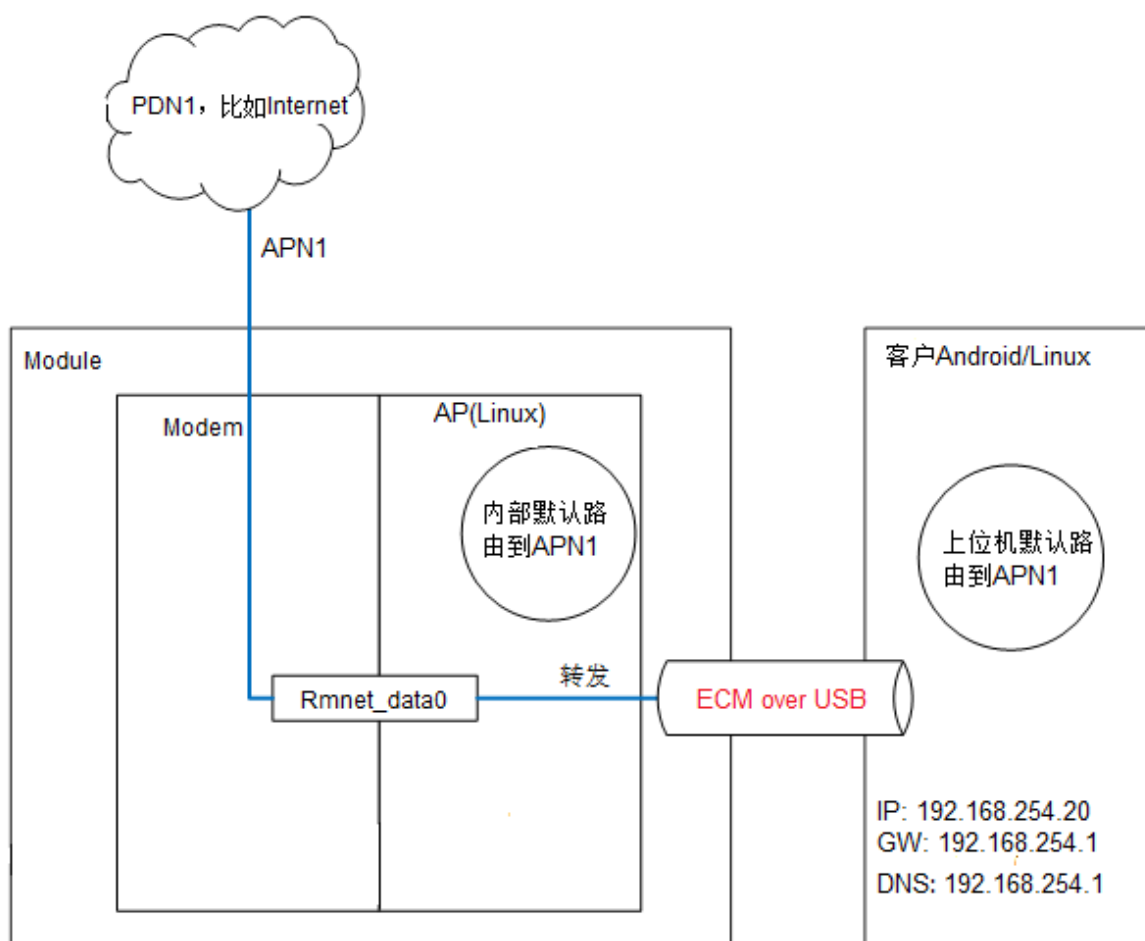


图 2：场景 2（单路 + ECM 设备）

此应用场景下无需配置任何路由、转发表和 DNS 配置，仅需按照第 3 章和第 3.3 章拨号步骤正确执行即可。

4.3. 场景 3：多路

本章节主要介绍多路拨号应用场景下的路由、转发表和 DNS 配置。

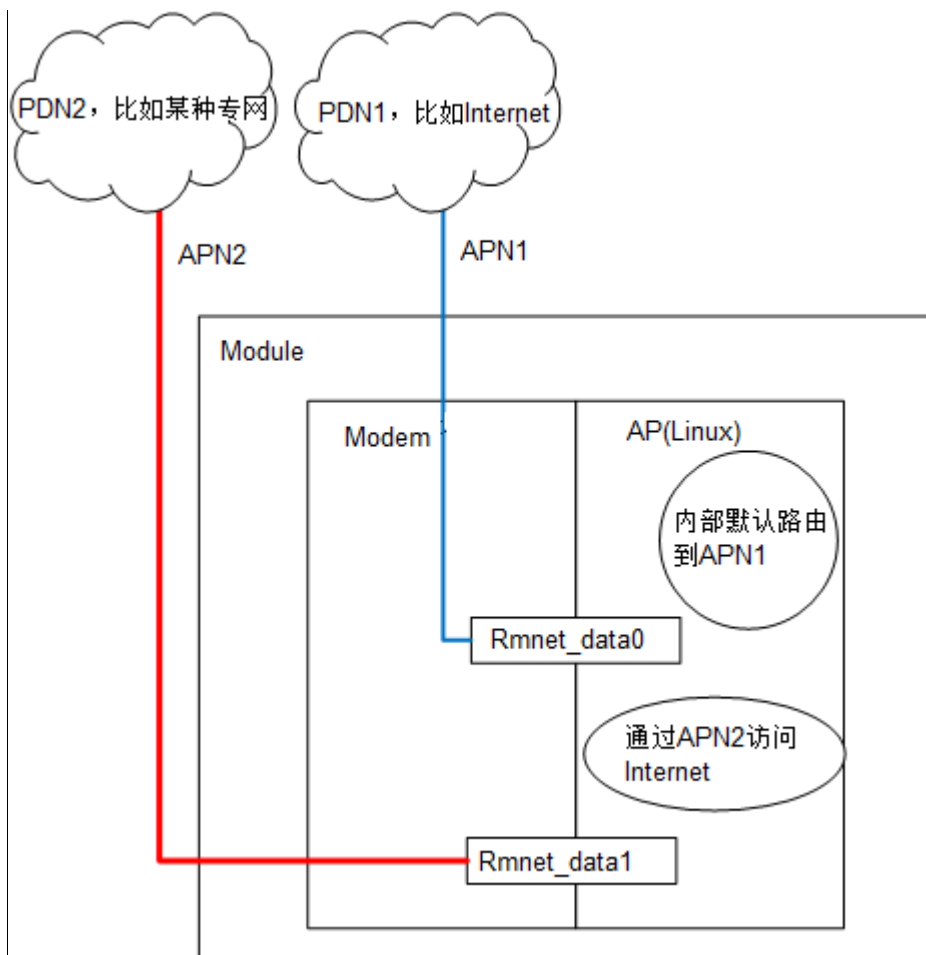


图 3：场景 3（多路）

4.3.1. 规则清空

```
route del default
iptables -t filter -F
iptables -t nat -F
```

4.3.2. 默认路由

```
//获取 APN1 网关地址。
Ql_Data_Call_Info_Get(2, &info);
```

//设置默认路由，假设获得的地址为 10.112.7.176。

方法一：

```
route add default dev rmnet_data0
```

方法二：

```
route add -net 10.112.7.0/24 dev rmnet_data0
```

```
route add default gw 10.112.7.176
```

4.3.3. 默认 DNS 设置

//获取 APN1 DNS 地址。

```
QI_Data_Call_Info_Get(1, &info);
```

//设置默认 DNS 服务器，假设获得的地址为 primary: 211.138.180.2 和 secondary: 211.138.180.3。

```
echo "nameserver 211.138.180.2" > /etc/resolv.conf
```

```
echo "nameserver 211.138.180.3" >> /etc/resolv.conf
```

4.3.4. 通过 APN2 访问某个 Server

4.3.4.1. DNS 路由配置

//获取 APN2 DNS 地址。

```
QI_Data_Call_Info_Get(2, &info);
```

//设置 DNS 路由，假设获得的地址为 primary: 121.158.280.8 和 secondary: 121.158.280.9。

```
ip route add 121.158.200.8/32 dev rmnet_data1
```

```
ip route add 121.158.200.9/32 dev rmnet_data1
```

4.3.4.2. 域名解析

//域名解析。

```
QL_nslookup( www.xxx.com, 121.158.280.8, IPV4, resolved_output)
```

其中 121.158.280.8 是 APN2 的 DNS 地址。

4.3.4.3. 路由配置

```
route add -net 47.88.189.189/32 gw 10.32.80.46 dev rmnet_data1
```

其中 47.88.189.189 是 www.xxx.com 地址，10.32.80.46 是 APN2 的网关（由 *QI_Data_Call_Info_Get* 获得）。

4.4. 场景 4：多路 + ECM（1）

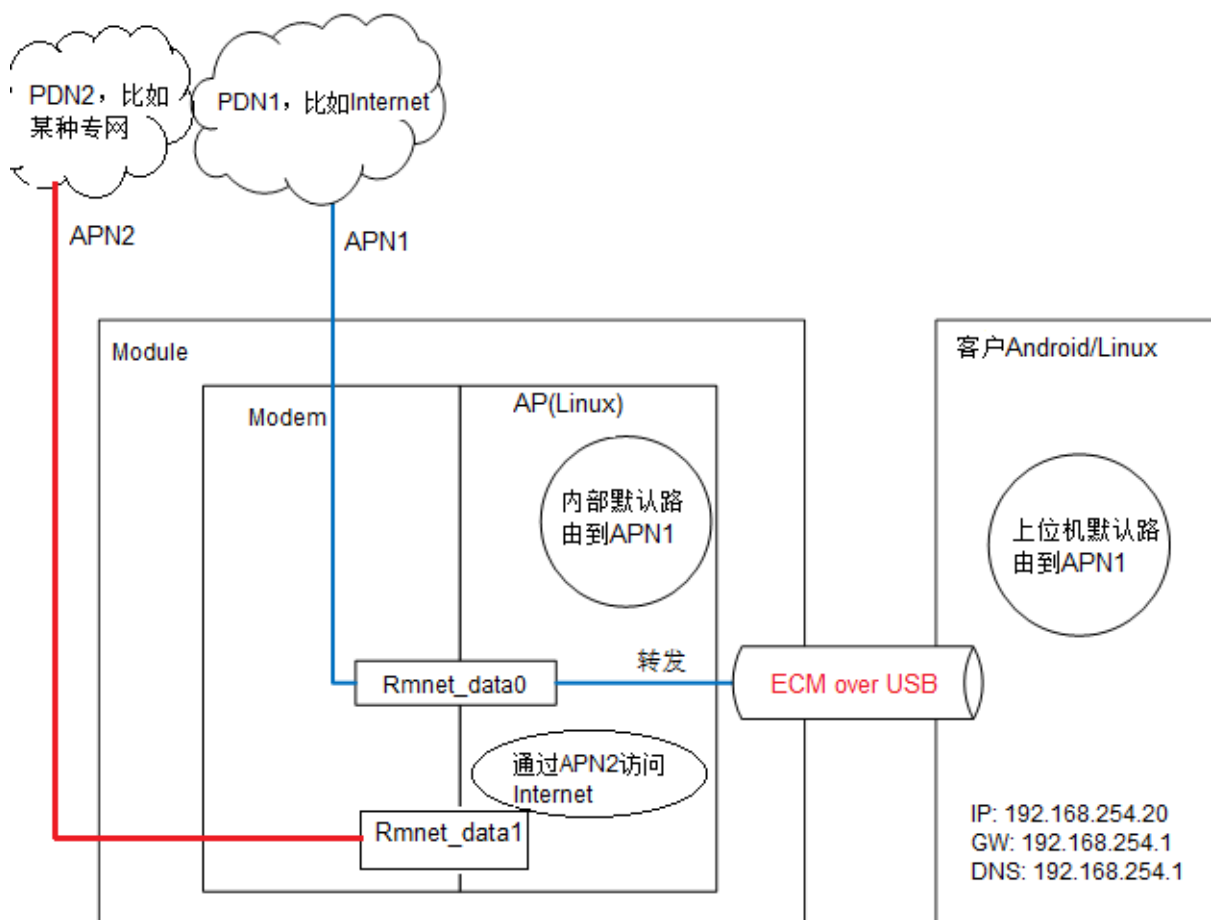


图 4：场景 4（多路 + ECM（1））

此应用场景下的基本步骤和**第 4.3 章**场景 3 基本一致，仅需在**第 4.3.4 章**之前打开默认转发表。

//开启默认转发。

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

//设置转发表。

```
iptables -t nat -A POSTROUTING -o rmnet_data0 -j MASQUERADE --random
```

4.5. 场景 5：多路 + ECM（2）

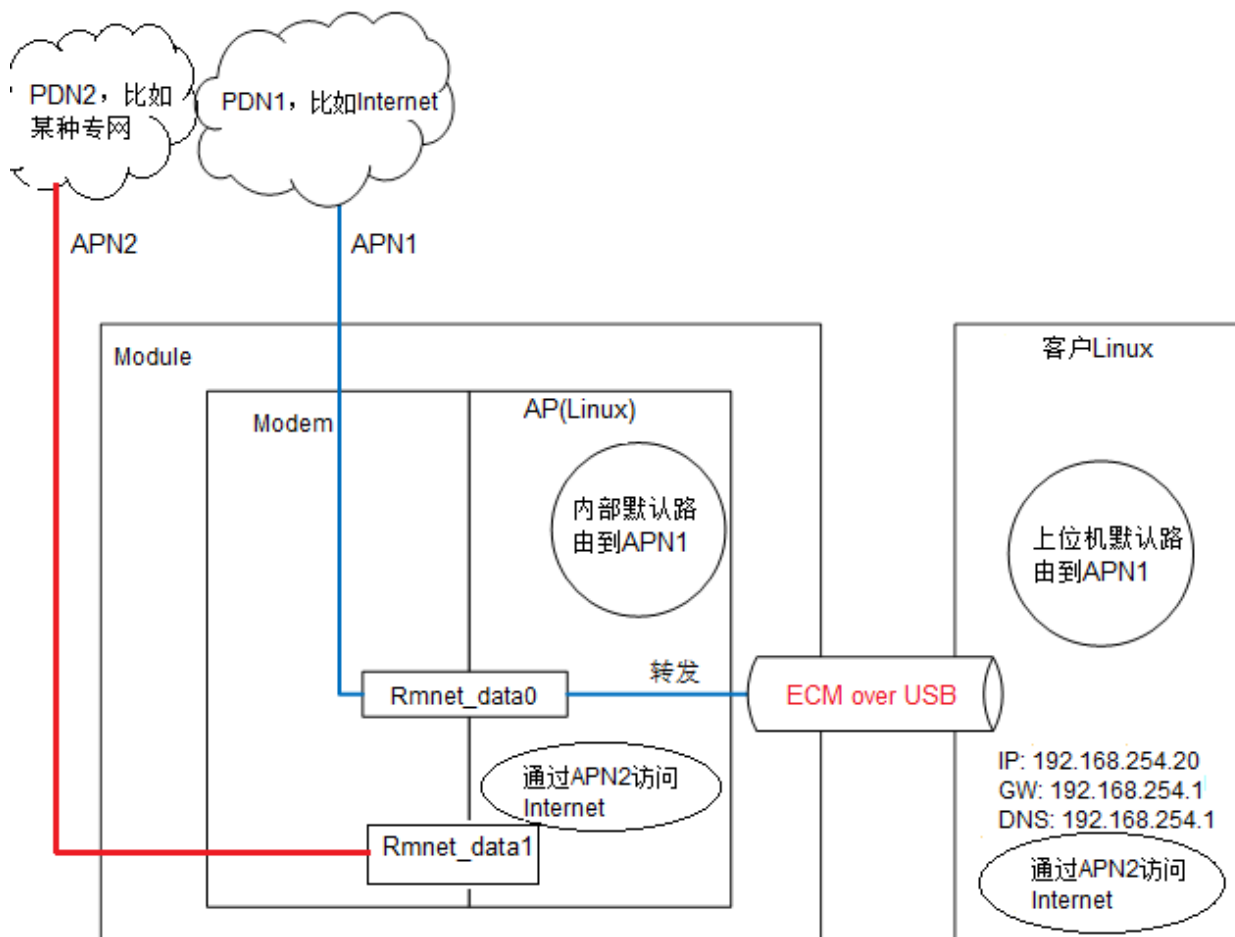


图 5：场景 5（多路 + ECM（2））

此应用场景下的基本步骤和**第 4.4 章**场景 4 基本一致。区别在于上位机通过 APN2 访问某些 Server，**第 4.5.1 和 4.5.2 章**为参考设计方案。

4.5.1. 域名解析

因上位机的域名解析全部由模块 AP 侧的 dnsmasq 通过 APN1 的 DNS 进行解析，故上位机想通过域名访问必须在 AP 侧建立一个 Server（一个 Socket）来完成 DNS 解析和转发表配置。

4.5.2. 路由设置

如果上位机直接基于 IP 地址访问，则只需要在 AP 侧配置一条转发规则，如下：

//假设 www.yyy.com 的 ip 地址为 47.88.189.189

//转发设置。

方法一：

```
iptables -t nat -A POSTROUTING -d 47.88.189.189 -o rmnet_data1 -j MASQUERADE
```

方法二：

```
iptables -t nat -A POSTROUTING -o rmnet_data1 -j MASQUERADE --random
```

//设置路由。

```
ip route add 47.88.189.189/32 dev rmnet_data1
```

4.6. 场景 6：多路 + ECM (3)

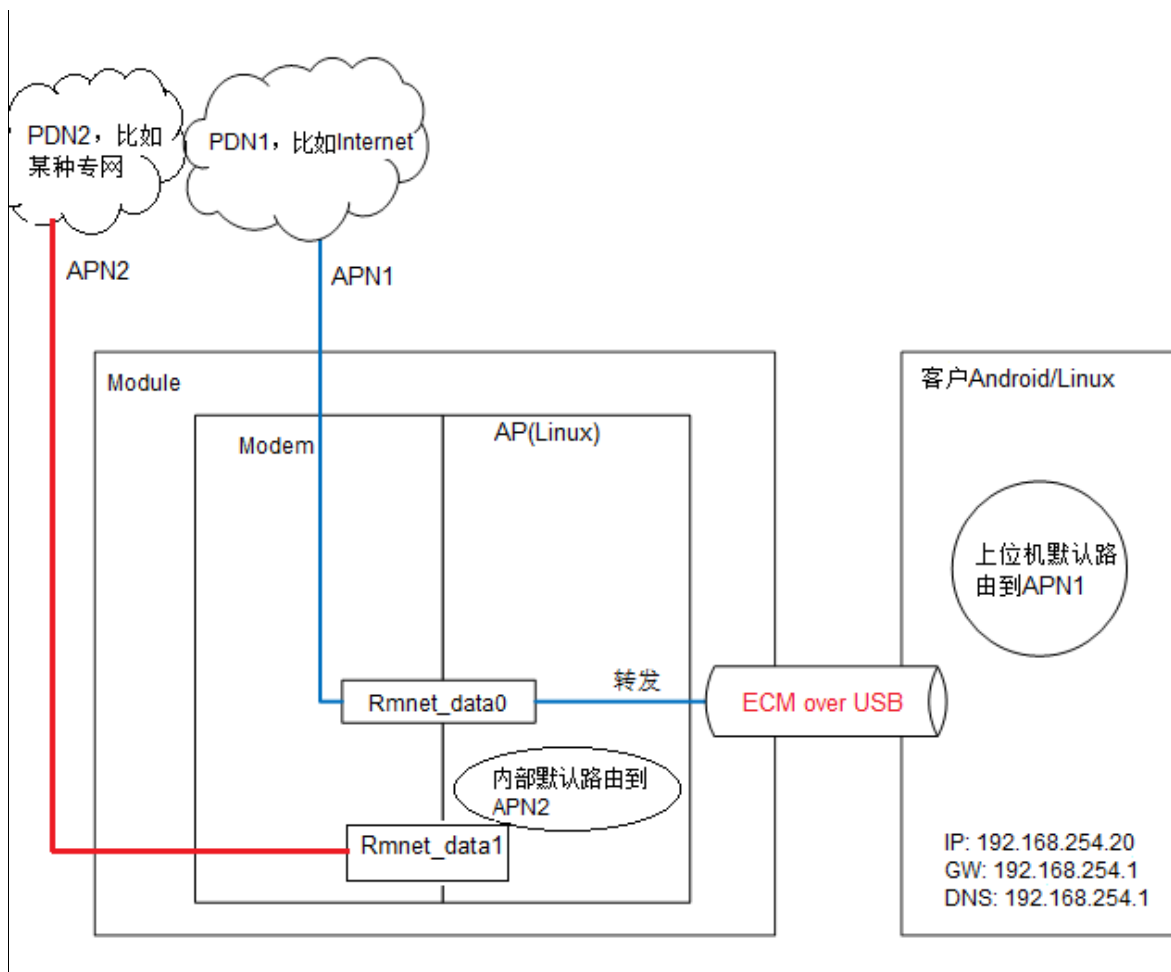


图 6：场景 6（多路 + ECM (3)）

4.6.1. 规则清空

```
route del default
iptables -t filter -F
iptables -t nat -F
```

4.6.2. AP 侧路由设置

```
//获取 APN2 网关地址。
Ql_Data_Call_Info_Get(2, &info);
//设置默认路由，假设获得的地址为 10.32.80.46。
方法一：
route add default dev rmnet_data1
方法二：
route add -net 10.32.80.0/24 dev rmnet_data1
route add default gw 10.32.80.46
```

4.6.3. AP 侧 DNS 设置

该步骤可实现模块内部默认 DNS 设置。

```
//获取 APN2 DNS 地址。
Ql_Data_Call_Info_Get(2, &info);

//设置默认 DNS 服务器(命令)，假设获得的地址为 primary: 121.158.280.8 和 secondary: 121.158.280.9。
echo "nameserver 121.158.280.8" > /etc/resolv.conf
echo "nameserver 121.158.280.9" >> /etc/resolv.conf
```

4.6.4. 转发表设置

设置转发表是实现上位机访问外网的关键步骤。

```
//开启默认转发。
echo 1 > /proc/sys/net/ipv4/ip_forward
//设置转发表。
iptables -t nat -A POSTROUTING -o rmnet_data0 -j MASQUERADE -random
```

4.6.5. 策略路由设置

在模块 AP 侧进行策略路由设置的操作步骤如下：

1. 执行如下命令创建路由表：

```
echo "200    rmnet_data_apn1 " >> /etc/iproute2/rt_tables
```

2. 执行如下命令设置策略路由：

```
ip rule add from 192.168.254.0/24 table 200    //192.168.254.0 为 ECM 设备的网络地址。
```

3. 执行如下命令添加路由规则：

方法一：

```
ip route add dev rmnet_data0 table 200
```

方法二：

```
ip route add via 10.112.7.176 table 200    //10.112.7.176 为网关地址。
```

4.6.6. 上位机 DNS 服务器设置

如果上位机 DNS 地址由模块下发，即借助模块使用基站下发 DNS 地址解析域名，则需要继续执行如下步骤。

如果上位机设定已知的公共 DNS 服务器，比如 8.8.8.8、114.114.114.114 等，则不需要执行如下步骤。

4.6.6.1. dnsmasq 配置

1. 修改/etc/dnsmasq.conf 文件。同步后重启模块。

```
# Change this line if you want dns to get its upstream servers from
# somewhere other than /etc/resolv.conf
resolv-file=/etc/dnsmasq_resolv.conf

# By default, dnsmasq will send queries to any of the upstream
# servers it knows about and tries to favour servers to are known
# to be up. Uncommenting this forces dnsmasq to try each query
# with each server strictly in the order they appear in
# /etc/resolv.conf
#strict-order
```

```
# If you don't want dnsmasq to read /etc/hosts, uncomment the
# following line.
no-hosts
# or if you want it to read another file, as well as /etc/hosts, use
# this.
#addn-hosts=/etc/banner_add_hosts
```

2. 创建/etc/dnsmasq_resolv.conf 文件，添加 APN1 的 DNS 服务器地址。

```
//获取 APN1 DNS 地址。
Ql_Data_Call_Info_Get(1, &info);
//假设获得的地址为 primary: 211.138.180.2 和 secondary: 211.138.180.3。
echo "nameserver 211.138.180.2" > /etc/ dnsmasq_resolv.conf
echo "nameserver 211.138.180.3" >> /etc/dnsmasq_resolv.conf
```

4.6.6.2. APN1 的 DNS 路由设置

```
//获取 APN1 DNS 地址。
Ql_Data_Call_Info_Get(1, &info);

//设置 DNS 路由，假设获得的地址为 primary: 211.138.180.2 和 secondary: 211.138.180.3。
ip route add 211.138.180.2/32 dev rmnet_data0
ip route add 211.138.180.3/32 dev rmnet_data0
```

4.7. 场景 7：多路 + ECM (4)

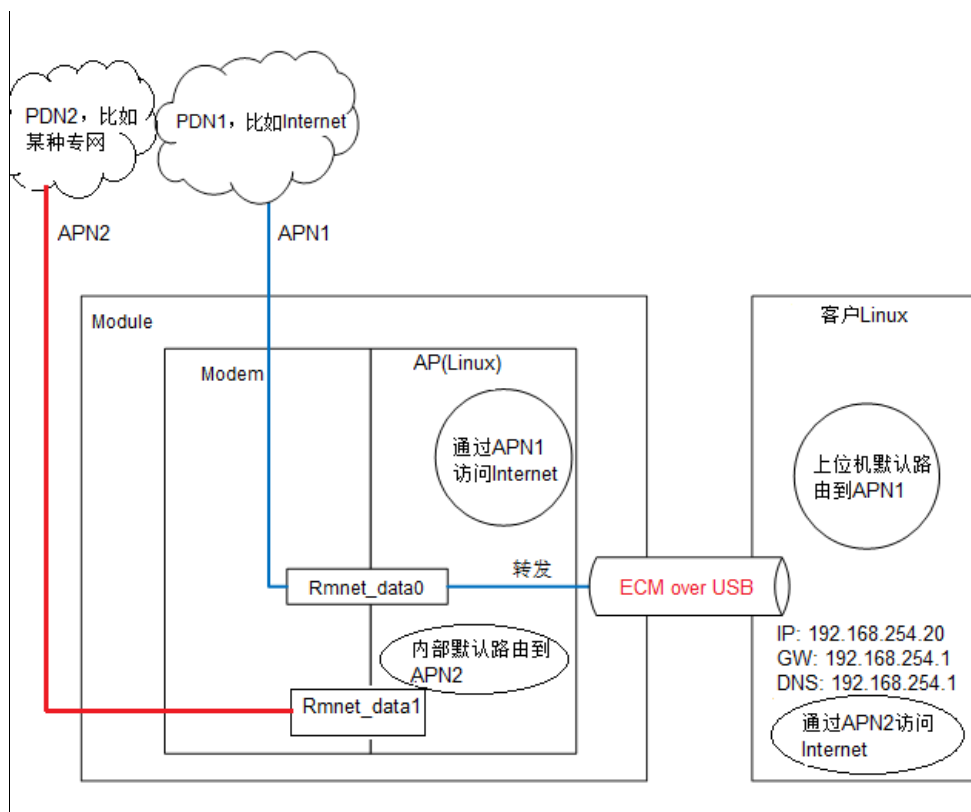


图 7：场景 7（多路 + ECM （4））

4.7.1. 基本设置

参照第 4.6 章场景 6 完成所有基本操作设置。

4.7.2. AP 侧通过 APN1 访问某个 Server

4.7.2.1. 域名解析

```
//域名解析。
QL_nslookup( www.xxx.com, 211.138.180.2, IPV4, resolved_output)
```

其中 211.138.180.2 是 APN1 的 DNS 地址。

4.7.2.2. 路由设置

```
//设置模块内部访问规则。
route add -net 47.88.189.189/32 gw 10.112.7.176 dev rmnet_data0
```

其中 47.88.189.189 是 www.xxx.com 地址，10.112.7.176 是 APN1 的网关（由 *Ql_Data_Call_Info_Get* 获得）。

4.7.3. 上位机通过 APN2 访问某个 Server

以下第 4.7.3.1 和 4.7.3.2 章为参考设计方案。

4.7.3.1. 域名解析

因上位机的域名解析全部由模块 AP 侧的 dnsmasq 通过 APN1 的 DNS 进行解析，即使使用公共 DNS 服务器也是如此。如上位机需要通过域名访问网络，则必须在 AP 侧建立一个 Server（一个 Socket）来完成 DNS 解析和转发表配置。

4.7.3.2. 路由设置

如果上位机直接基于 IP 地址访问，则只需要在 AP 侧配置一条转发规则，如下：

```
//假设 www.yyy.com 的 ip 地址为 47.88.189.189
//设置转发。
方法一：
iptables -t nat -A POSTROUTING -d 47.88.189.189 -o rmnet_data1 -j MASQUERADE
方法二：
```

```
iptables -t nat -A POSTROUTING -o rmnet_data1 -j MASQUERADE --random
```

//设置路由。

```
ip route add 47.88.189.189/32 dev rmnet_data1 table 200
```

//或基于策略路由设置路由。

```
ip rule add to 47.88.189.189 table main
```

4.8. 场景 8：多路 + ECM (5)

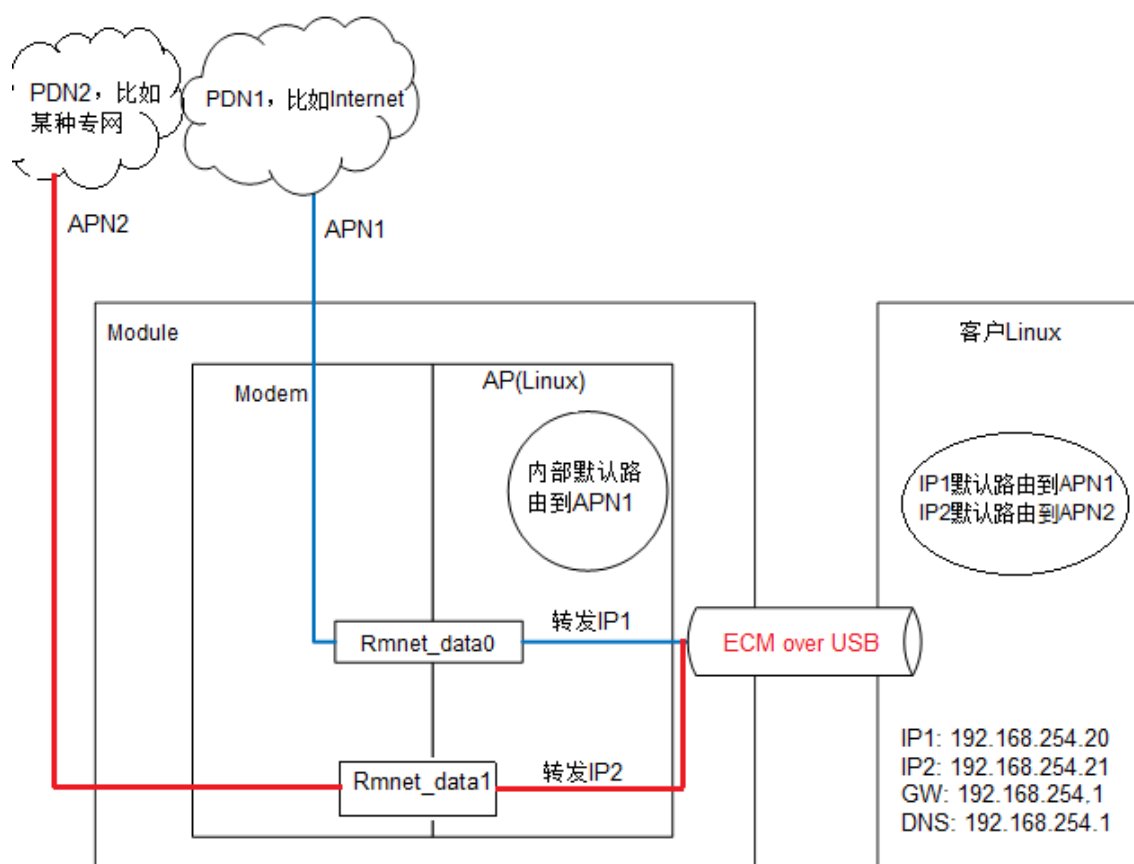


图 8：场景 8（多路 + ECM (5)）

4.8.1. AP 侧路由和 DNS 设置

参照第 4.6.1 章和第 4.6.3 章。

4.8.2. 上位机策略路由设置

参照第 4.6.4 章和第 4.6.5 章。

4.8.3. DNS 解析设置

DNS 解析设置要遵循各自 APN 对应下发的 DNS 地址，策略路由表里面可创建如下路由规则：

1. 访问 APN1 下发的 DNS 服务器地址时，DNS 服务器地址路由到 APN1 网关。
2. 访问 APN2 下发的 DNS 服务器地址时，DNS 服务器地址路由到 APN2 网关。

5 补充说明

本章对 Linux 系统的 DNS 作出一些额外说明，以及对专网域名解析给出指导意见。

5.1. API

1. Linux 标准函数 `gethostbyname()` 使用 `resolve.conf` 里面的 DNS 地址进行 IPv4 地址解析，或者从 `/etc/hosts` 直接解析。
2. Linux 标准函数 `getaddrinfo()` 使用 `resolve.conf` 里面的 DNS 地址同时进行 IPv4 和 IPv6 地址解析，或者从 `/etc/hosts` 直接解析。
3. 移远通信函数 `QL_nslookup()` 使用指定的 DNS 地址能够分别单独解析 IPv4 和 IPv6 地址。

备注

以上 API 都不能保证一次性解析成功，需要反复调用 2~3 次，这是因为 DNS 协议传输层采用 UDP 协议，丢失报文是很正常的现象。另外 DNS 服务器本身的网络连通性也是很重要。

5.2. 命令

5.2.1. PING

执行此命令后，返回的域名解析基于 Linux 系统默认 DNS。

5.2.2. NSLOOKUP

执行此命令，模块只能使用 `resolve.conf` 默认的 DNS 进行解析，与在 PC 上面使用该命令有差异。

6 数据拨号 API

6.1. 数据拨号 API 类型

6.1.1. ql_data_call_error_e

```
typedef enum {
    QL_DATA_CALL_ERROR_NONE = 0,
    QL_DATA_CALL_ERROR_INVALID_PARAMS,
} ql_data_call_error_e;
```

6.1.2. ql_data_call_state_e

```
typedef enum {
    QL_DATA_CALL_DISCONNECTED = 0,          /*!< call is disconnected */
    QL_DATA_CALL_CONNECTED,                 /*!< call is connected */
} ql_data_call_state_e;
```

6.1.3. ql_data_call_ip_family_e

```
typedef enum {
    QL_DATA_CALL_TYPE_IPV4 = 0,             /*!< IPv4 call. */
    QL_DATA_CALL_TYPE_IPV6,                 /*!< IPv6 call. */
    QL_DATA_CALL_TYPE_IPV4V6,               /*!< IPv4 and IPv6 call (Only used call start or stop). */
} ql_data_call_ip_family_e;
```

6.1.4. ql_apn_pdp_type_e

```
typedef enum {
    QL_APN_PDP_TYPE_IPV4 = 0,
    QL_APN_PDP_TYPE_PPP,
    QL_APN_PDP_TYPE_IPV6,
    QL_APN_PDP_TYPE_IPV4V6,
} ql_apn_pdp_type_e;
```

6.1.5. ql_apn_auth_proto_e

```
typedef enum {
    QL_APN_AUTH_PROTO_DEFAULT = 0,
    QL_APN_AUTH_PROTO_NONE,
    QL_APN_AUTH_PROTO_PAP,
    QL_APN_AUTH_PROTO_CHAP,
    QL_APN_AUTH_PROTO_PAP_CHAP,
} ql_apn_auth_proto_e;
```

6.1.6. v4_address_status

```
struct v4_address_status {
    struct in_addr ip;                /*!< Public IPv4 address. */
    struct in_addr gateway;          /*!< Public IPv4 gateway. */
    struct in_addr pri_dns;           /*!< Primary Domain Name Service IP address. */
    struct in_addr sec_dns;          /*!< Secondary Domain Name Service IP address. */
};
```

6.1.7. v6_address_status

```
struct v6_address_status {
    struct in6_addr ip;              /*!< Public IPv6 address. */
    struct in6_addr gateway;         /*!< Public IPv6 gateway. */
    struct in6_addr pri_dns;         /*!< Primary Domain Name Service IPv6 address. */
    struct in6_addr sec_dns;         /*!< Secondary Domain Name Service IPv6 address. */
};
```

6.1.8. ql_data_call_state_s

```
typedef struct {
    char profile_idx;                /*!< UMTS/CMDA profile ID. */
    char name[16];                   /*!< Interface Name. */
    ql_data_call_ip_family_e ip_family; /*!< IP version. */
    ql_data_call_state_e state;       /*!< The dial status. */
    ql_data_call_error_e err;         /*!< The Reason code after data call disconnected. */

    union {
        struct v4_address_status v4; /*!< IPv4 information. */
        struct v6_address_status v6; /*!< IPv6 information. */
    };
} ql_data_call_state_s;
```

6.1.9. ql_data_call_s

```

/*
 *!< Client callback function used to post event indications. */
typedef void (*ql_data_call_evt_cb_t)(ql_data_call_state_s *state);

typedef struct {
    char profile_idx;                /*!< UMTS/CMDA profile ID. */
    bool reconnect;                  /*!< Whether to re-dial after disconnecting the network. */
    ql_data_call_ip_family_e ip_family; /*!< IP version. */

    char cdma_username[127];
    char cdma_password[127];
} ql_data_call_s;

```

6.1.10. pkt_stats

```

struct pkt_stats {
    unsigned long pkts_tx;           /*!< Number of packets transmitted. */
    unsigned long pkts_rx;           /*!< Number of packets received. */
    long long bytes_tx;              /*!< Number of bytes transmitted. */
    long long bytes_rx;              /*!< Number of bytes received. */
    unsigned long pkts_dropped_tx;   /*!< Number of transmit packets dropped. */
    unsigned long pkts_dropped_rx;   /*!< Number of receive packets dropped. */
};

```

6.1.11. v4_info

```

struct v4_info {
    char name[16];                   /*!< Interface Name. */
    ql_data_call_state_e state;      /*!< The dial status. */
    bool reconnect;                  /*!< re-dial flag. */
    struct v4_address_status addr;    /*!< IPv4 IP Address information. */
    struct pkt_stats stats;           /*!< IPv4 statics */
};

```

6.1.12. v6_info

```

struct v6_info {
    char name[16];                   /*!< Interface Name. */
    ql_data_call_state_e state;      /*!< The dial status. */
    bool reconnect;                  /*!< re-dial flag. */
    struct v6_address_status addr;    /*!< IPv6 IP Address information. */
};

```

```
struct pkt_stats stats;                                /*!< IPv6 statics */
};
```

6.1.13. ql_data_call_info_s

```
typedef struct {
    char profile_idx;                                /*!< UMTS/CDMA profile ID. */
    ql_data_call_ip_family_e ip_family;              /*!< IP version. */
    struct v4_info v4;                               /*!< IPv4 information */
    struct v6_info v6;                               /*!< IPv6 information */
} ql_data_call_info_s;
```

6.1.14. ql_apn_info_s

```
typedef struct {
    unsigned char profile_idx;                        /*!< UMTS/CDMA profile ID. */
    ql_apn_pdp_type_e pdp_type;
    ql_apn_auth_proto_e auth_proto;                  /*!< Authentication Protocol. */
    char apn_name[QL_APN_NAME_SIZE];
    char username[QL_APN_USERNAME_SIZE];             /*!< Username used during data network
authentication. */
    char password[QL_APN_PASSWORD_SIZE];             /*!< Password to be used during data network
authentication. */
} ql_apn_info_s;
```

6.1.15. ql_apn_add_s

```
typedef struct {
    ql_apn_pdp_type_e pdp_type;
    ql_apn_auth_proto_e auth_proto;                  /*!< Authentication Protocol. */
    char apn_name[QL_APN_NAME_SIZE];
    char username[QL_APN_USERNAME_SIZE];             /*!< Username used during data network
authentication. */
    char password[QL_APN_PASSWORD_SIZE];             /*!< Password to be used during data network
authentication. */
} ql_apn_add_s;
```

6.1.16. ql_apn_info_list_s

```
typedef struct {
    int cnt;
    ql_apn_info_s apn[QL_APN_MAX_LIST];
} ql_apn_info_list_s;
```

6.2. 函数

6.2.1. QL_Data_Call_Init

该函数用于初始化模块数据拨号，并注册回调函数。

- 函数原型

```
int QL_Data_Call_Init (ql_data_call_evt_cb_t evt_cb)
```

- 参数说明

evt_cb:

[In] 回调函数

- 返回值

0: 执行成功

-1: 执行失败

6.2.2. QL_Data_Call_Destroy

该函数用于释放模块数据拨号，并去注册回调函数。

- 函数原型

```
void QL_Data_Call_Destroy (void)
```

- 参数说明

无

- 返回值

0: 执行成功

-1: 执行失败

6.2.3. QL_Data_Call_Start

该函数用于开始数据拨号。

- 函数原型

```
int QL_Data_Call_Start (ql_data_call_s *data_call, ql_data_call_error_e *err)
```

- 参数说明

data_call:

[In] 数据拨号参数

error:

[Out] 数据拨号返回的错误码

- 返回值

0: 执行成功

-1: 执行失败

6.2.4. QL_Data_Call_Stop

该函数用于结束数据拨号。

- 函数原型

```
int QL_Data_Call_Stop (char profile_idx, ql_data_call_ip_family_e ip_family, ql_data_call_error_e *err)
```

- 参数说明

profile_idx:

[In] UMTS/CDMA 文件 ID

ip_family:

[In] IP 版本

error:

[Out] 数据拨号返回的错误码

- 返回值

0: 执行成功

-1: 执行失败

6.2.5. QL_Data_Call_Info_Get

该函数用于获取数据拨号信息。

- 函数原型

```
int QL_Data_Call_Info_Get (
char profile_idx,
ql_data_call_ip_family_e ip_family,
ql_data_call_info_s *info,
ql_data_call_error_e *err)
```

- 参数说明

profile_idx:

[In] UMTS/CDMA 文件 ID

ip_family:

[In] IP 版本

Info:

[Out] 数据拨号信息

error:

[Out] 数据拨号返回的错误码

- 返回值

0: 执行成功

-1: 执行失败

6.2.6. QL_APN_Set

该函数用于更改配置文件中的设置。若文件不存在，将创建新的配置文件，且该文件带有使用此 API 配置的设置。

- 函数原型

```
int QL_APN_Set (ql_apn_info_s *apn)
```

- 参数说明

apn:

[In] 文件信息

- 返回值
- 0: 执行成功
-1: 执行失败

6.2.7. QL_APN_Get

该函数用于读取配置文件中的设置。

- 函数原型

```
int QL_APN_Get (unsigned char profile_idx, ql_apn_info_s *apn)
```

- 参数说明

profile_idx:

[In] UMTS/CDMA 文件 ID

apn:

[Out] 文件信息

- 返回值
- 0: 执行成功
-1: 执行失败

6.2.8. QL_APN_Add

该函数用于添加一路 APN 实例。

- 函数原型

```
int QL_APN_Add(ql_apn_add_s *apn, unsigned char *profile_idx)
```

- 参数说明

profile_idx:

[Out] UMTS/CDMA 文件 ID

apn:

[In] 文件信息

- 返回值
- 0: 执行成功
-1: 执行失败

6.2.9. QL_APN_Del

该函数用于删除配置文件。

- 函数原型

```
int QL_APN_Del(unsigned char profile_idx)
```

- 参数说明

profile_idx:

[In] UMTS/CDMA 文件 ID

- 返回值

0: 执行成功

-1: 执行失败

6.2.10. QL_APN_Get_Lists

该函数用于读取配置文件列表中的设置。

- 函数原型

```
int QL_APN_Get_Lists(ql_apn_info_list_s *apn_list)
```

- 参数说明

apn_list:

[Out] 文件列表信息

- 返回值

0: 执行成功

-1: 执行错误

6.2.11. QL_Data_Call_Init_Precondition

该函数用于获取 Quectel Manager 服务的运行状态。

- 函数原型

```
QL_Data_Call_Init_Precondition
```

- 参数说明

无

- 返回值

0: 服务工作正常

-1: 不正常

7 常见问题

7.1. 抓取 PCAP 日志

1. 输入如下命令抓取所需 log。

抓取所有网口 PCAP 日志：

```
tcpdump -i any -p -vv -s 0 -w ./capture1.pcap &
```

抓取指定网口 PCAP 日志：

```
tcpdump -i rmnet_data0 -p -vv -s 0 -w ./capture1.pcap &
```

2. 运行程序。
3. 中断 tcpdump 命令，上传 *capture1.pcap* 文件。

7.2. 查看 iptables 表

1. 执行以下命令查看 nat 表。

```
iptables -nvt nat -L
```

2. 执行以下命令查看 filter 表。

```
iptables -nvt filter -L
```

7.3. 3GPP2/CDMA 因鉴权无法拨号上网

请按照如下步骤解决 3GPP2/CDMA 因为鉴权无法拨号上网的问题。

1. Modem 侧检查

参数设置：

```
AT+QCTPWDCFG="<username>","<userpasswd>"
AT+QCFG="cdmaruim",1
```

拨号测试：

```
AT+QIACT=1 //失败返回 ERROR，多数是因为用户名和密码不正确。
```

2. AP 侧检查

如果第一步检查无任何问题，需检查 `profile_id` 是否为 0，鉴权参数是否正确，具体参照第 3.2 章和第 3.5 章。

7.4. 模块开机拨号失败

模块开机自动运行客户 APP 程序，会概率出现拨号失败的问题。检查发现如果注网等各项都正常，此时再次进行拨号，拨号成功。

该问题主要原因为拨号时其服务进程尚未启动完成，可通过 `QL_Data_Call_Init_Precondition()` 查询服务是否已启动完成。

7.5. 发送 DNS Server IPv6 Address Request 失败

模块在采用 IPv4&IPv6 拨号时，在当前网络不支持 IPv6 的情况下可能会出现发送 DNS Server IPv6 Address Request 失败的问题。

针对此问题，目前的解决方案是在拨号时指定仅采用 IPv4 进行拨号。具体操作为在调用 `QL_Data_Call_Start` 函数时，指定 `ql_data_call_s` 中的 `ip_family` 为 `QL_DATA_CALL_TYPE_IPV4`。

8 附录 A 参考文档及术语缩写

表 2: 参考文档

序号	文档名称	备注
[1]	Quectel_EC2x&EG9x&EG25-G 系列_QuecOpen _ECM_用户指导	适用于 EC2x&EG9x&EG25-G 系列 QuecOpen 的 USB-ECM 用户指导
[2]	Quectel_EC2x&EG9x&EG25-G 系列_QuecOpen _RNDIS_用户指导	适用于 EC2x&EG9x&EG25-G 系列 QuecOpen 的 USB-RNDIS 用户指导

表 3: 术语缩写

术语	英文全称	中文全称
3GPP2	3rd Generation Partnership Project 2	第三代合作伙伴计划 2
API	Application Programming Interface	应用程序编程接口
APN	Access Point Name	接入点名称
APP	Application	应用程序
CDMA	Code Division Multiple Access	码分多址
DNS	Domain Name System	域名系统
ECM	Ethernet Networking Control Model	以太网控制模型
eHRPD	Evolved High Rate Package Data	演进的高速分组网络
EPS	Evolved Packet System	演进的分组系统
HRPD	High Rate Packet Data	高速率分组数据
IPv4	Internet Protocol version 4	网际协议版本 4
IPv6	Internet Protocol version 6	网际协议版本 6

LTE	Long Term Evolution	长期演进
PDP	Packet Data Protocol	分组数据协议
RNDIS	Remote Network Driver Interface Specification	远程网络驱动接口规范
SDK	Software Development Kit	软件开发工具包
SIM	Subscriber Identification Module	用户身份识别卡
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发传输器
(U)SIM	(Universal) Subscriber Identification Module	（全球）用户身份识别卡
USB	Universal Serial Bus	通用串行总线
UMTS	Universal Mobile Telecommunications System	通用移动通信系统